

Travail sur le microcontrôleur

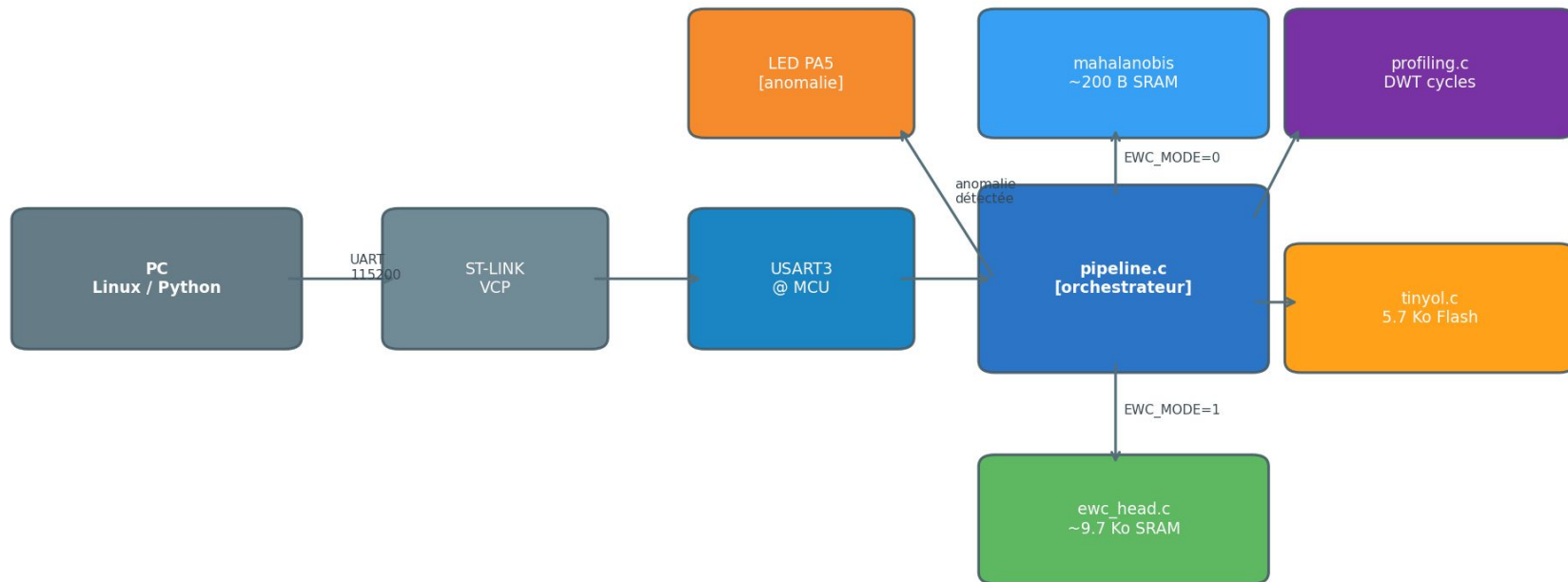
Juin 2026 _ Leonard Rivals
ISAE-SUPAERO / ENAC / Edge Spectrum

Sommaire

- Lien PC et Carte
- Protocole de communication ordinateur carte
- Les scripts Python côté PC
- Les modèles de Continual Learning portés en C
- Profiling hardware
- Analyse des résultats des expériences
- Pistes pour la suite

Lien PC et Carte

Architecture firmware NUCLEO-F439ZI — vue d'ensemble



Lien PC et Carte

```
firmware/stm32f4_blink/
```

```
├─ src/
```

```
├─ main.c           ← point d'entrée, init hardware  
├─ pipeline.c       ← orchestrateur UART + inférence  
├─ ewc_head.c       ← MLP + EWC (modèle M2)  
├─ tinyol.c         ← autoencodeur TinyOL (modèle M1)  
└─ profiling.c      ← mesure latence + RAM via DWT
```

```
├─ inc/
```

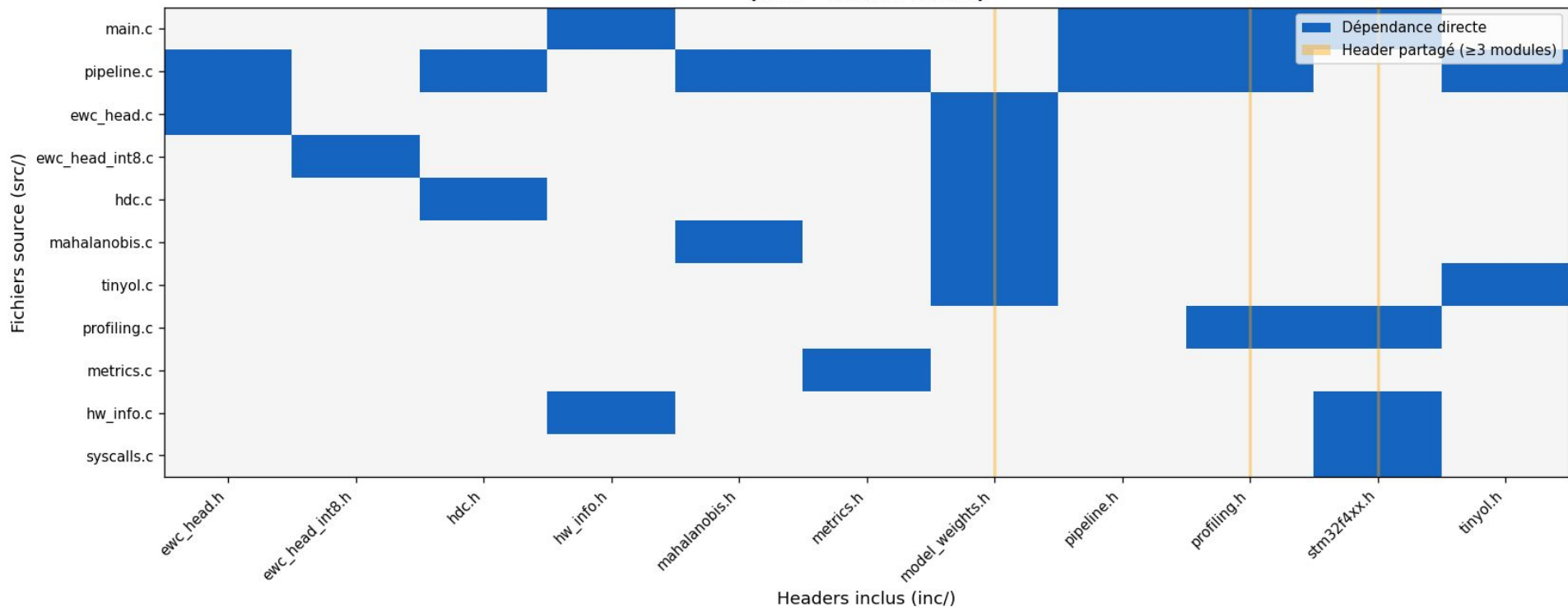
```
├─ pipeline.h       ← protocoles, flags, macros LED  
├─ ewc_head.h       ← structs + constantes EWC  
├─ tinyol.h         ← structs + constantes TinyOL  
├─ profiling.h      ← ProfilingState struct  
└─ model_weights.h ← poids en Flash (constantes)
```

```
└─ tests/
```

```
├─ test_runner.c    ← orchestre tous les tests Unity  
├─ test_models.c    ← tests Mahalanobis + EWC + TinyOL + métriques  
├─ test_ewc_head.c  ← 17 tests ciblés EWC  
└─ mock_data.h      ← données synthétiques de test
```

Lien PC et Carte

**Sprint 16 – Matrice de dépendances firmware NUCLEO-F439ZI
(bleu = header inclus)**



Sprint 17 — Protocole UART : Diagramme de séquence PC ↔ MCU (6 étapes séquentielles par cycle de 32 octets reçus)



Protocole de communication ordinateur carte

Sprint 17 — Flags de contrôle du pipeline firmware

Flags de contrôle — pipeline.c

0x01**UPDATE**

Mettre à jour les poids du modèle avec ce sample

0x02**CONSOLIDATE**

Calculer Fisher + star_weights (fin de tâche EWC)

0x04**RESET**

Réinitialiser le modèle aux poids initiaux

0x08**EWC_MODE**

Activer la régularisation EWC pendant l'update

0x00**QUERY_ONLY**

Inférence seule, aucune mise à jour des poids

0x10**METRICS_RESET**

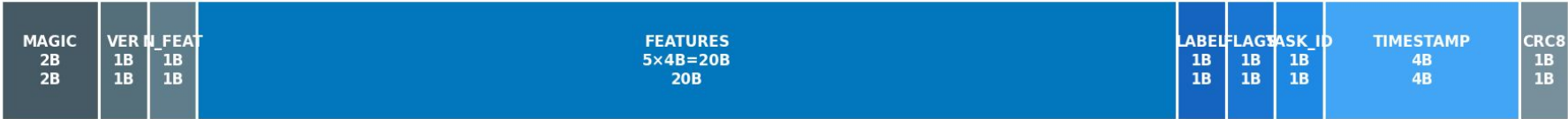
Remettre à zéro les compteurs de métriques CL

Combinaisons de flags courantes

	UPD	CON	RST	EWC
0x00 Inférence seule	0	0	0	0
0x01 Train + inférence	1	0	0	0
0x09 Train + EWC	1	0	0	1
0x0A Fin tâche (consolidate)	0	1	0	1
0x04 Reset modèle	0	0	1	0
0x10 Reset métriques	0	0	0	0

Évolution du protocole UART binaire v2→v3

Trame PC → Carte (request) (32 octets total)



Réponse Carte → PC — Protocol v2 (Sprint 18) (14 octets total)



Réponse Carte → PC — Protocol v3 (Sprint 19, +métriques CL) (19 octets total)



Protocole de communication ordinateur carte

Trame UART v2 — Réponse (14 octets, little-endian)

Octet(s)	Champ	Type	Description
0	PRED	uint8	Prédiction (0=normal, 1=anomalie)
1-2	CONFIDENCE	uint16	Confiance × 10 000 (ex : 8754 = 87,54%)
3-6	LATENCY_US	uint32	Latence inférence en microseconde
7-8	RAM_B	uint16	Taille .bss en bytes
9-10	THROUGHPUT	uint16	Inférences/seconde
11	STATUS	uint8	Code d'erreur (0=OK)
12-13	CRC16	uint16	CRC sur les 12 premiers octets

Extension du Protocole de communication ordinateur carte

Trame UART v3 — Champs supplémentaires vs v2 (octets 14-20, métriques CL temps réel)

Octet(s)	Champ	Type	Description
14-15	ACC	uint16	Accuracy × 10 000 (online, depuis c
16-17	AUROC	uint16	AUROC × 10 000 (estimation glissa
18-19	FORGETTING	uint16	Forgetting × 10 000 (chute max ac
20	CRC8	uint8	CRC sur les 20 premiers octets

Les scripts Python côté PC

`sensor_sim.py` — Simulateur simple:

Envoie des samples d'un dataset au format protocole v2.

`sensor_stream.py` — Streaming multi-tâches avancé

Script principal pour les expériences CL. Gère :

- Le découpage domain-incremental (`--cl-sequence "pump:167,turbine:167,compressor:166"`)
- L'envoi des flags de consolidation à la fin de chaque tâche
- La réception et décodage des réponses v2/v3
- Le rate-limiting (ex: `--hz 10` = 10 samples/seconde maximum)
- L'export des résultats (latences, accuracies, profiling)

Les scripts Python côté PC

`board_experiment_recorder.py` — Enregistreur d'expériences

Gère une expérience complète et génère un `results.json` compatible avec le format Phase 1 (`evaluate_all.py`). Calcule automatiquement les métriques CL.

Métriques calculées automatiquement :

- `acc_final` : accuracy globale sur toutes les tâches après entraînement complet
- `avg_forgetting` (AF) : chute moyenne d'accuracy entre le pic et la fin par tâche
- `backward_transfer` (BWT) : impact de l'apprentissage futur sur les tâches passées
- `ram_peak_bytes` : RAM .bss mesurée sur la carte
- `inference_latency_ms` : latence DWT en millisecondes

Les modèles de Continual Learning portés en C

Mahalanobis — Le baseline statistique

- * mesurer si un nouveau point est loin de la distribution normale apprise.
- * tient compte des corrélations entre features :
 - μ : vecteur moyen
 - Σ^{-1} : matrice de précision (inverse de covariance)

Amélioration : Adaptation en ligne du vecteur moyen (EMA)

Limites : modèle linéaire (pas de non-linéarités), sensible aux outliers dans l'EMA.

Les modèles de Continual Learning portés en C

EWC Head MLP — CL avec régularisation

réseau de neurones (MLP) qui protège les poids importants pour les tâches précédentes.

La matrice Fisher diagonale F estime l'importance de chaque poids pour les tâches précédentes.

Les poids importants (F élevé) sont protégés via la régularisation

Les modèles de Continual Learning portés en C

TinyOL — Autoencoder pour détection d'anomalie

un autoencoder (encoder + decoder) est pré-entraîné sur des données normales.
Lors du déploiement, si la reconstruction d'un sample est mauvaise (MSE élevée),
c'est une anomalie.

Les modèles de Continual Learning portés en C

HDC — Hyperdimensional Computing

pas de gradient, pas de rétropropagation. Tout est basé sur des opérations sur des vecteurs binaires de haute dimension ($D=1000$ bits).

les prototypes de classe sont des accumulateurs. Apprendre un nouveau sample = additionner son hypervecteur au prototype. Les tâches précédentes restent intactes.

Limitation : pas d'adaptation fine (le prototype est la somme de tous les samples vus, pas un vrai gradient descent).

Profiling hardware

Objectif: démontrer du continual learning avec moins de 256 Ko de RAM et des chiffres mesurés précisément sur hardware réel.

Mesure de la RAM — `parse_map_file.py`

Après compilation, le linker génère un fichier `.map` qui liste tous les symboles et leurs tailles et script `parse_map_file.py` l'analyse

Profiling Hardware NUCLEO-F439ZI – Sections linker et symboles clés

Sections mémoire analysées (linker map)

Section	Emplacement	Rôle
.text	Flash	Code compilé (ne consomme pas de SRAM)
.rodata	Flash	Constantes en Flash (poids TinyOL, z-score stat)
.data	SRAM (copié de Flash)	Variables initialisées
.bss	SRAM	Variables non-initialisées (poids EWC, Mahalanobis)
.ccmram	CCM RAM (64 Ko)	RAM CCM (utilisée pour le stack)

Symboles clés en .bss (poids modèles)

Symbole	Section	Taille	Description
g_detector	.bss	~200 B	struct Mahalanobis
g_ewc_head	.bss	~9.7 Ko	struct EWCHHead
g_tinyol_enc / g_tinyol_dec	.bss	~400 B	structs TinyOL enc + dec
g_profiling	.bss	~20 B	ProfilingState

Profiling hardware

Mesure de la latence — DWT

Le DWT (Data Watchpoint and Trace) est un compteur de cycles hardware précis à 1 cycle (≈ 5.5 ns @ 180 MHz)

Cette valeur est ensuite encodée dans la réponse UART et récupérée par le PC pour être enregistré dans profiling.json

Analyse des résultats des expériences

Experiences on Board

Données réelles issues des datasets existants, mais pas le dataset complet.

extrait $n_samples$ échantillons découpés en n_tasks tâches

-> sélection séquentielle des données existantes

La carte reçoit les features réels, fait son inférence en C , et renvoie sa prédiction.

Analyse des résultats des expériences

CWRU streaming board

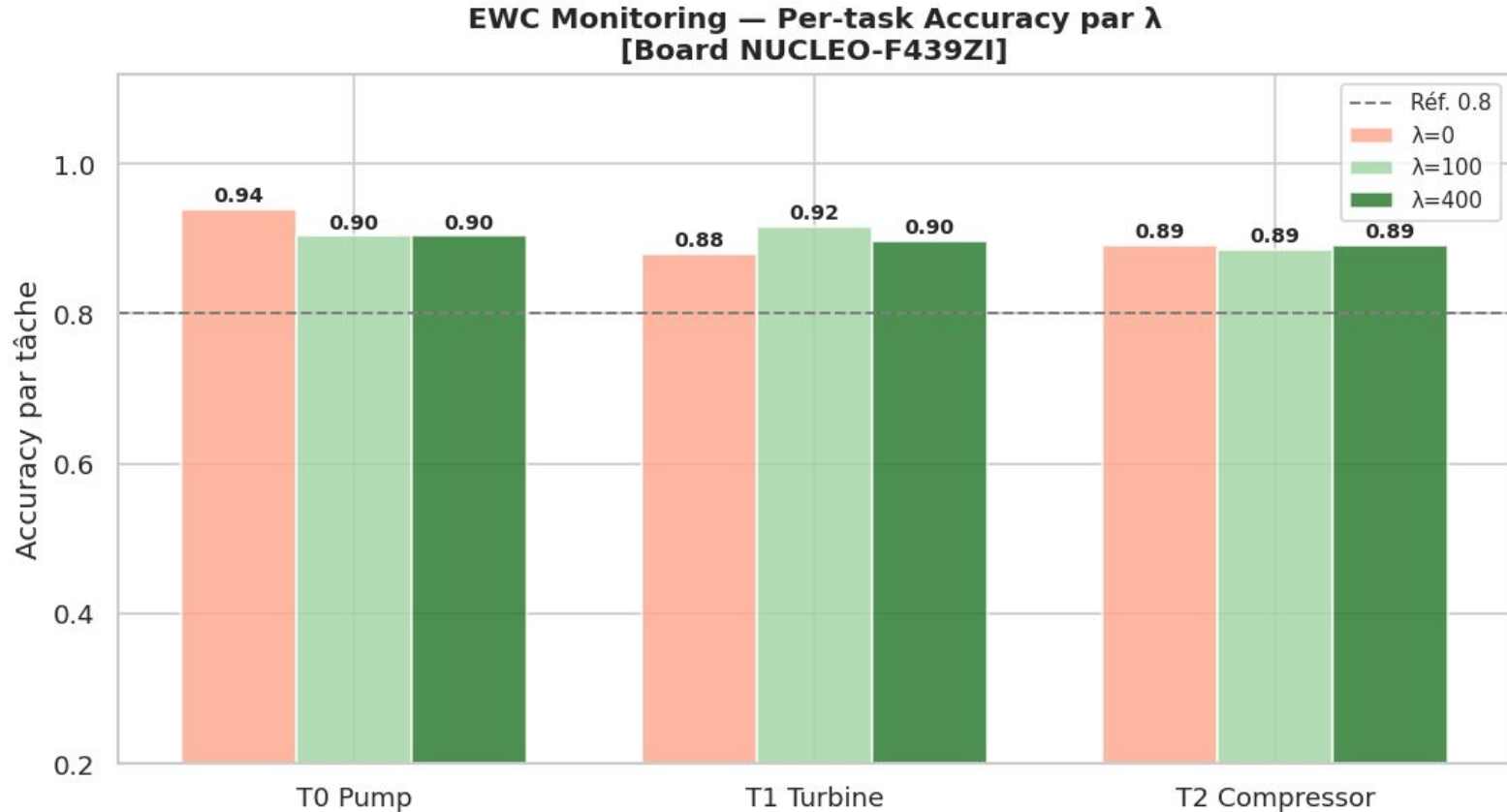
dataset CWRU, 498 samples, 3 tâches, modèle Mahalanobis

Métrique	Valeur mesurée	Budget Gap 2	Marge
RAM .bss	1 000 B	< 64 Ko	×64
Latence moyenne	3.7 μ s	< 100 ms	×27 000
Latence P99	4.0 μ s	< 100 ms	×25 000
Throughput	34 235 ips	—	—

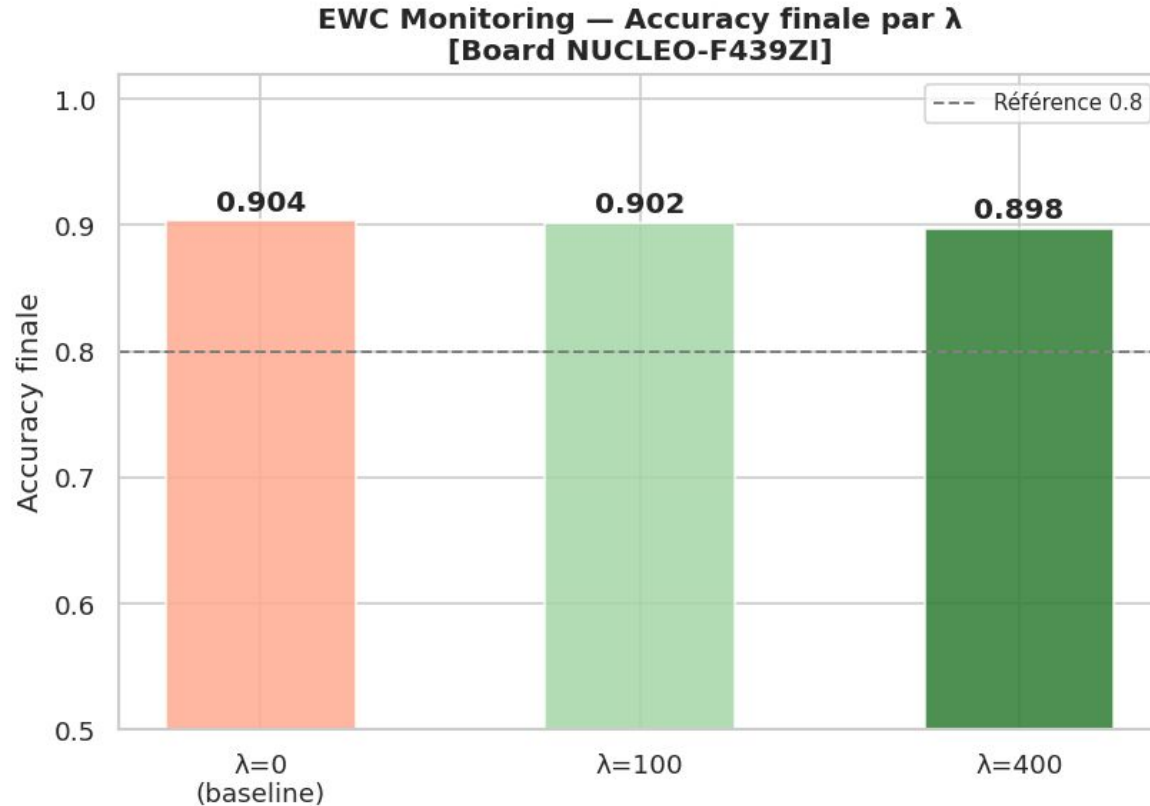
Analyse des résultats des expériences

EWC 3 tâches Monitoring Test de l'impact du paramètre λ
Monitoring 500 samples, 3 tâches, $\lambda=[0; 100; 400]$

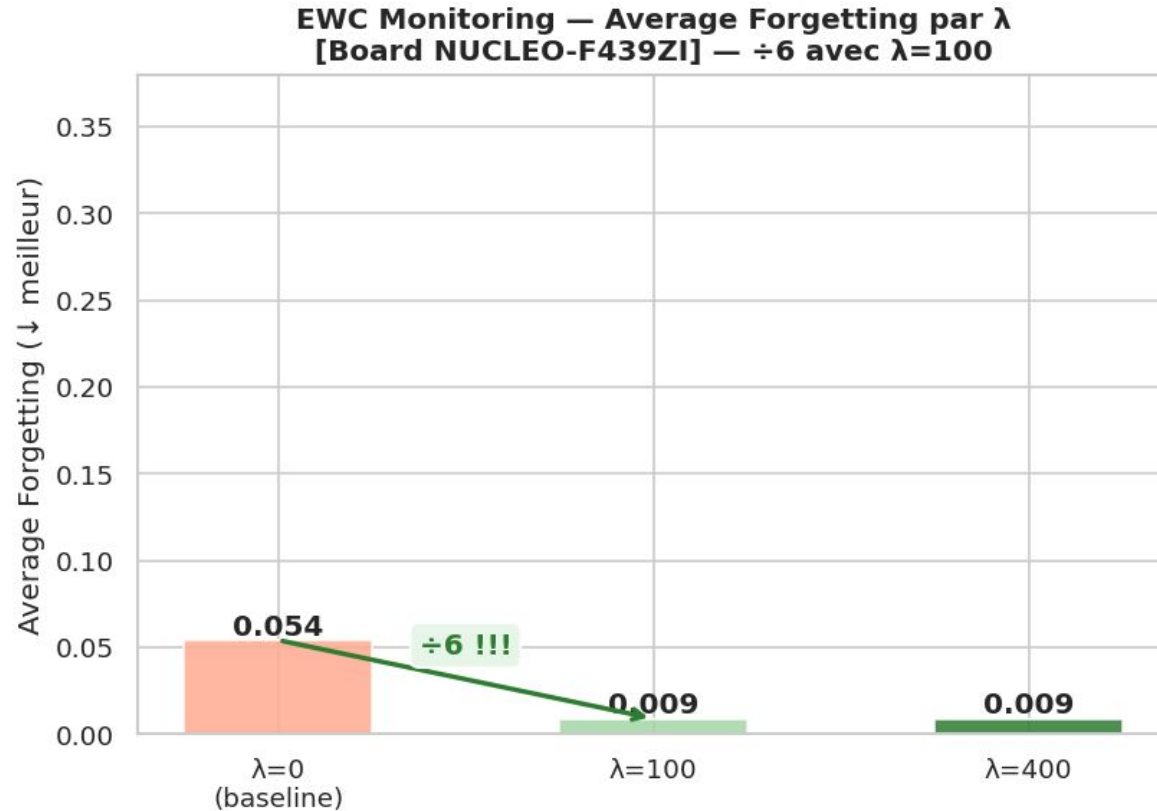
Analyse des résultats des expériences



Analyse des résultats des expériences



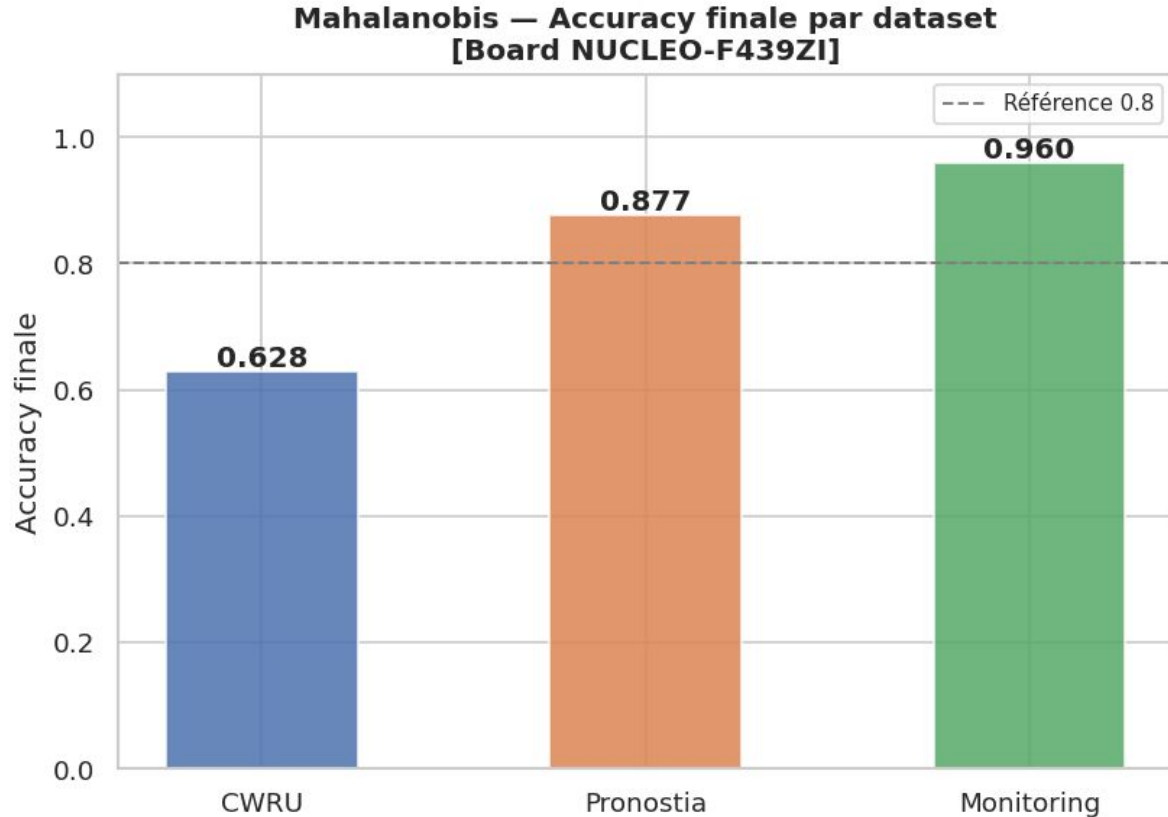
Analyse des résultats des expériences



Analyse des résultats des expériences

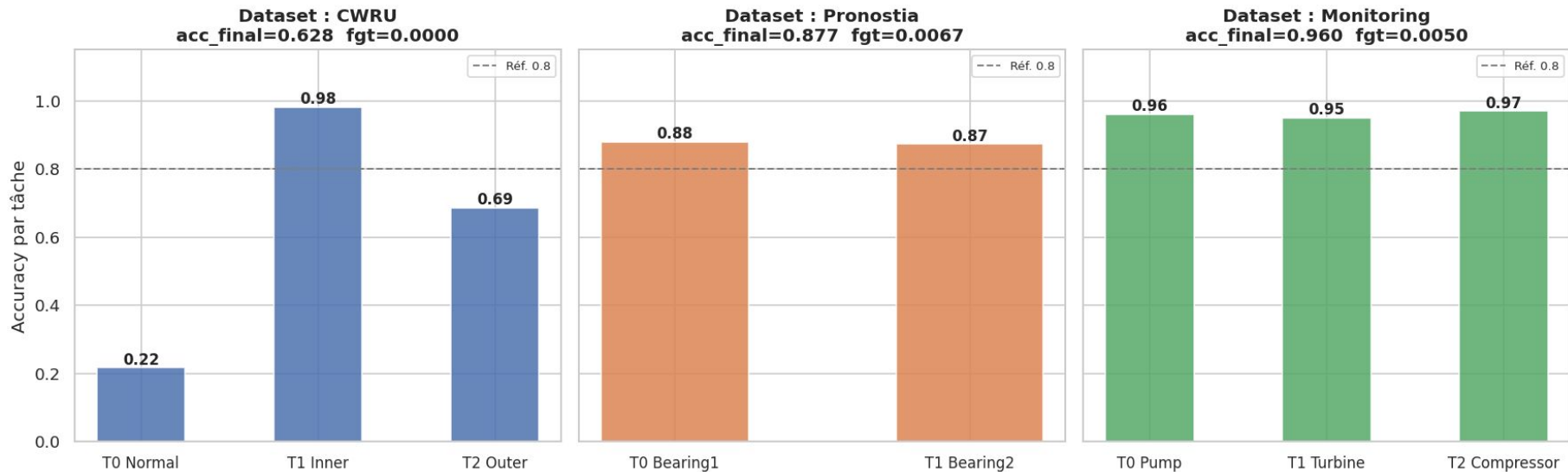
Evaluation des modèles Mahalanobis, EWC, HDC et TinyOL
Sur les datasets CWRU, Pronostia, Monitoring

Analyse des résultats des expériences _ Mahalanobis

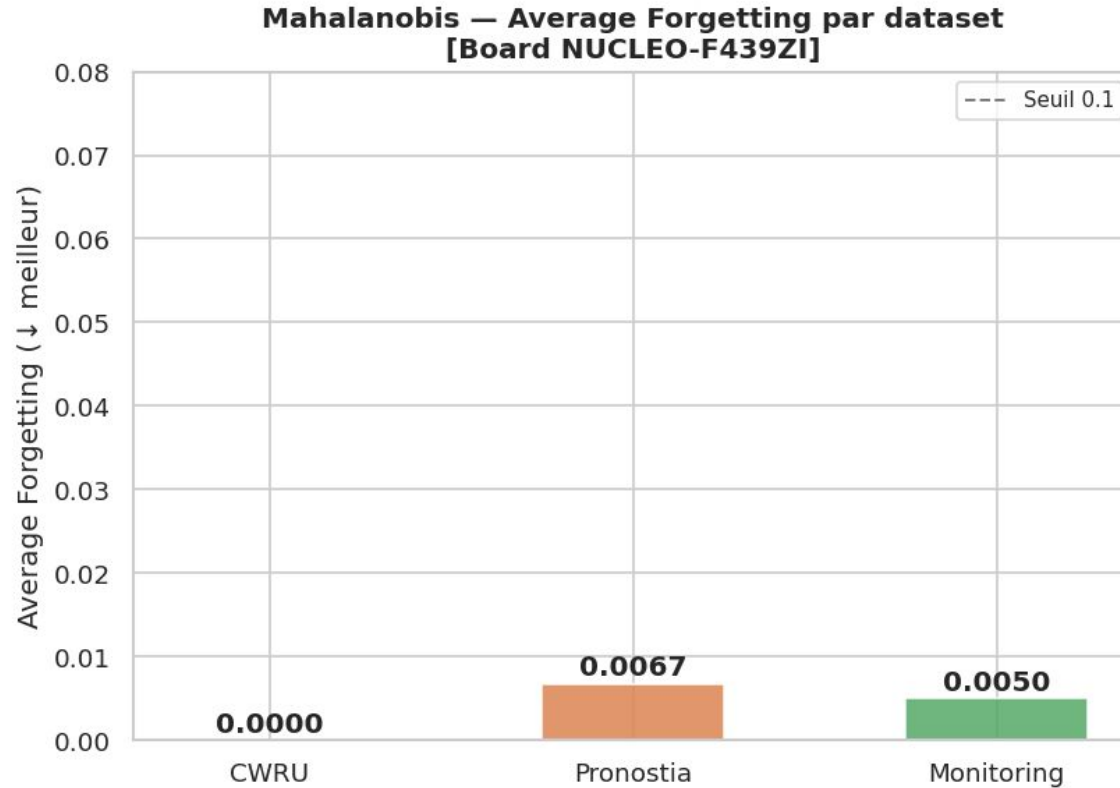


Analyse des résultats des expériences _ Mahalanobis

Mahalanobis C — Per-task Accuracy par dataset [Board NUCLEO-F439ZI]



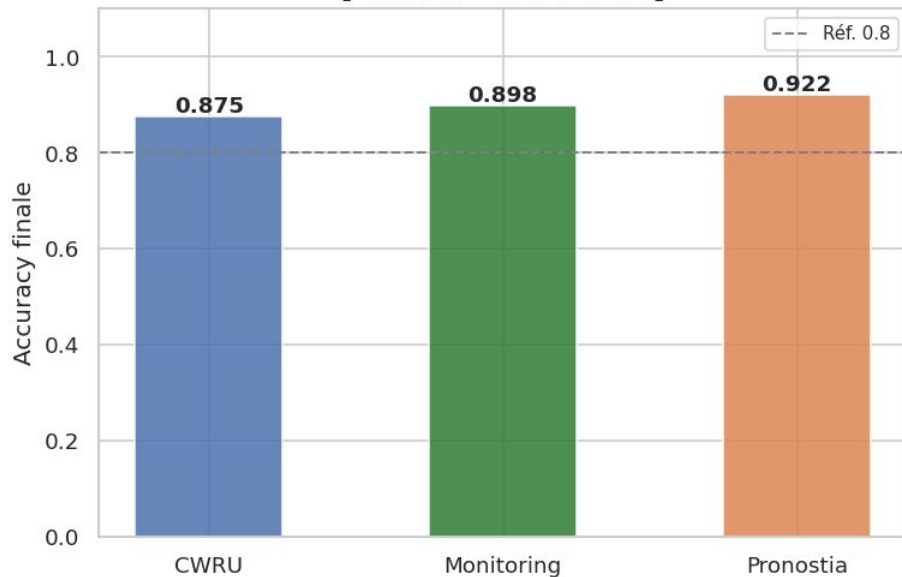
Analyse des résultats des expériences _ Mahalanobis



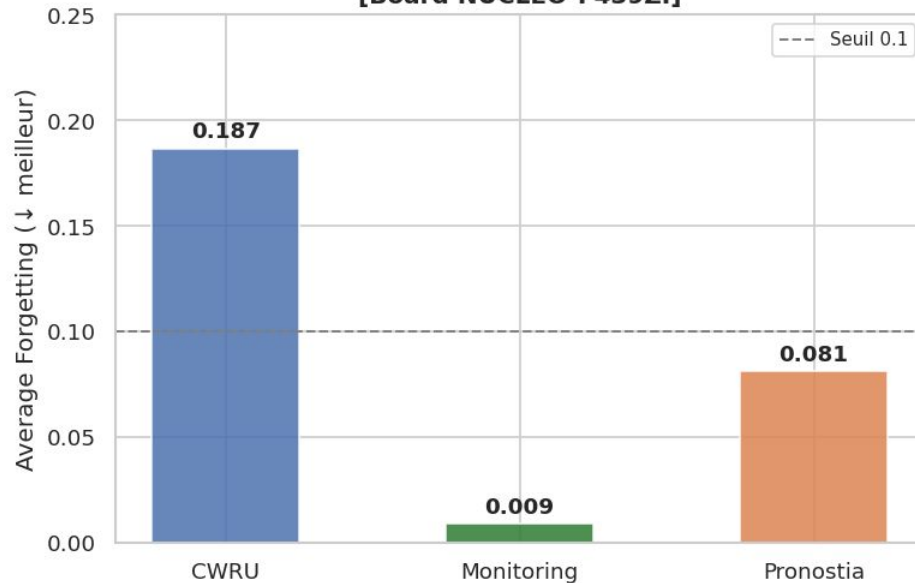
Analyse des résultats des expériences _ EWC

Sprint 19 — EWC $\lambda=400$ cross-dataset (board) — exp_S19_05 / S19_02 / S19_06

EWC $\lambda=400$ — Accuracy finale par dataset
[Board NUCLEO-F439ZI]



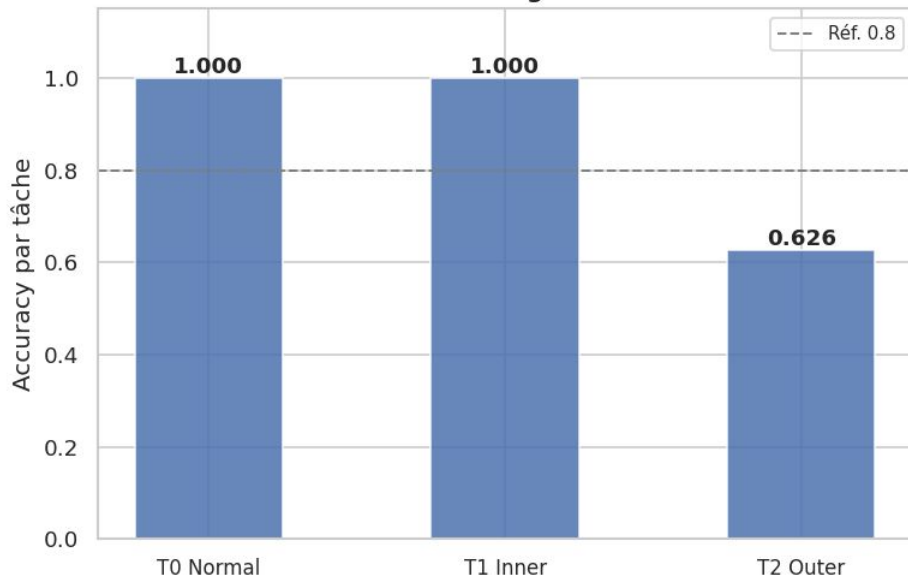
EWC $\lambda=400$ — Average Forgetting par dataset
[Board NUCLEO-F439ZI]



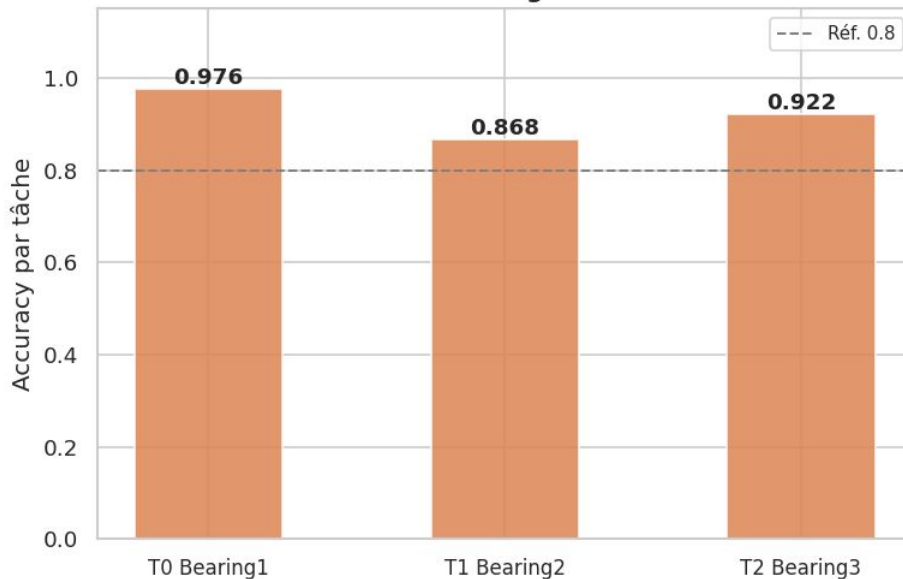
Analyse des résultats des expériences _ EWC

Sprint 19 — EWC $\lambda=400$: per-task accuracy CWRU + Pronostia (board)

EWC $\lambda=400$ — Per-task (CWRU)
acc=0.875 fgt=0.187



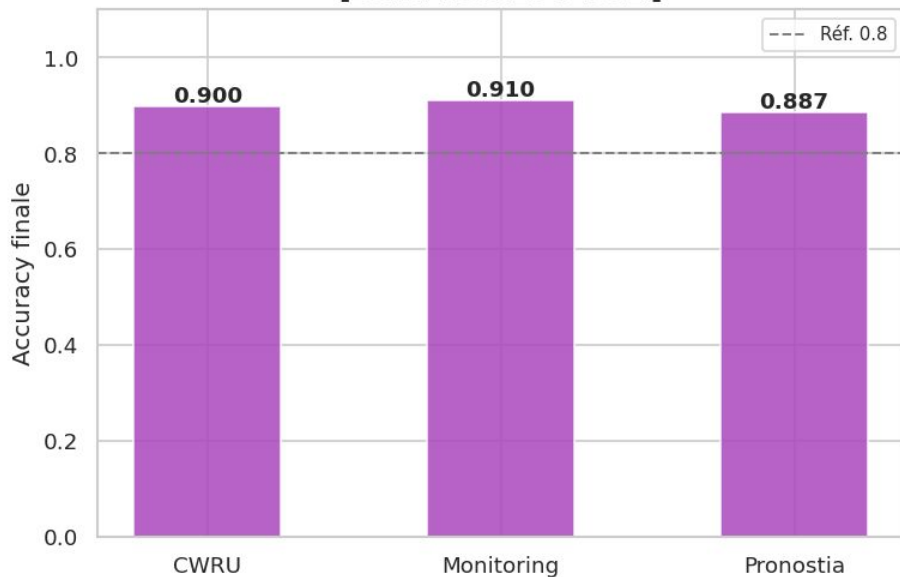
EWC $\lambda=400$ — Per-task (Pronostia)
acc=0.922 fgt=0.081



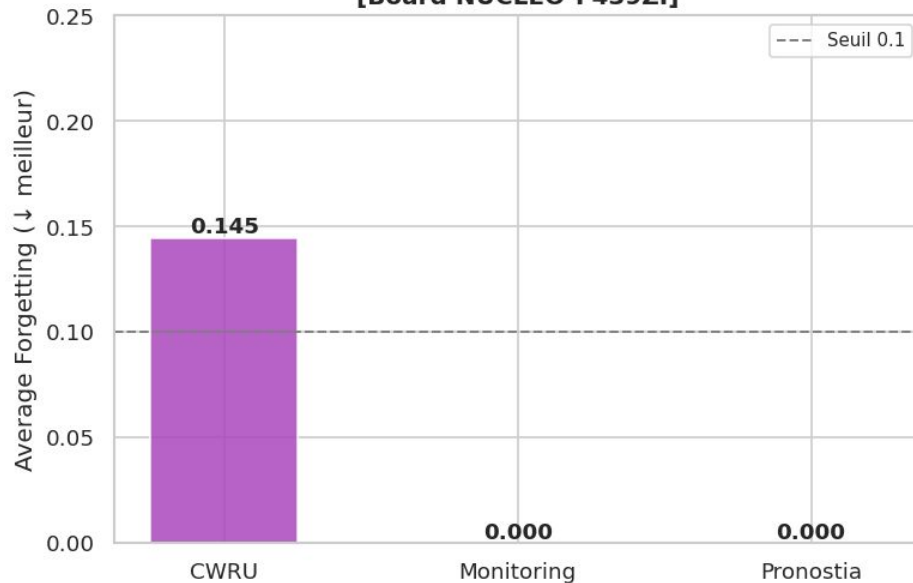
Analyse des résultats des expériences _HDC

Sprint 19 — HDC cross-dataset (board) — exp_S19_07 / S19_08 / S19_09

HDC — Accuracy finale par dataset
[Board NUCLEO-F439ZI]

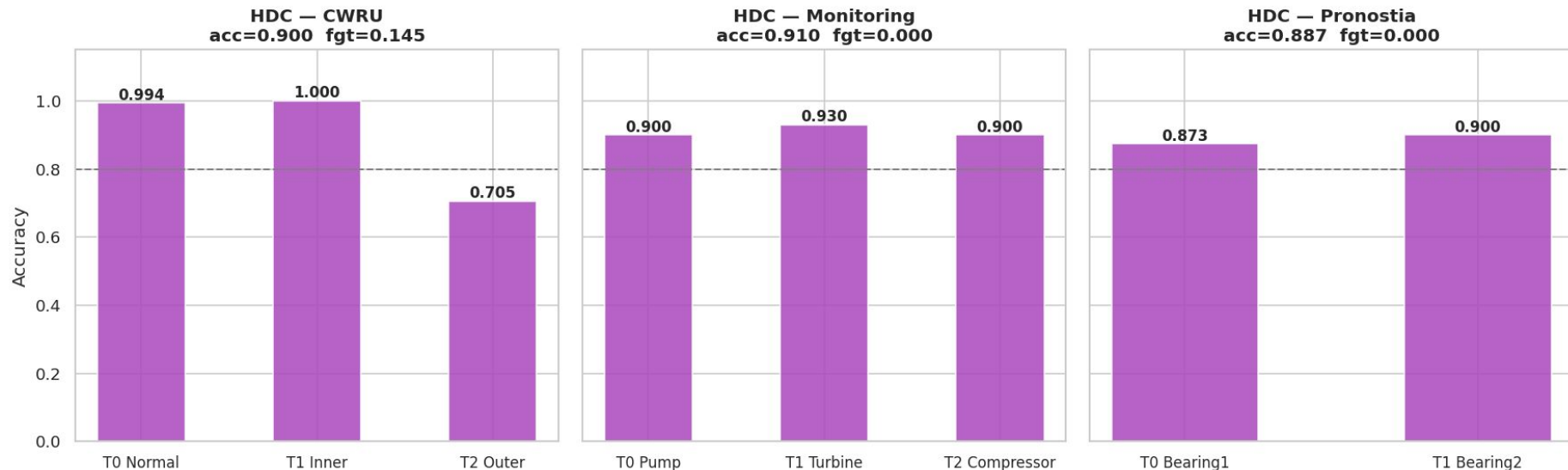


HDC — Average Forgetting par dataset
[Board NUCLEO-F439ZI]



Analyse des résultats des expériences _ HDC

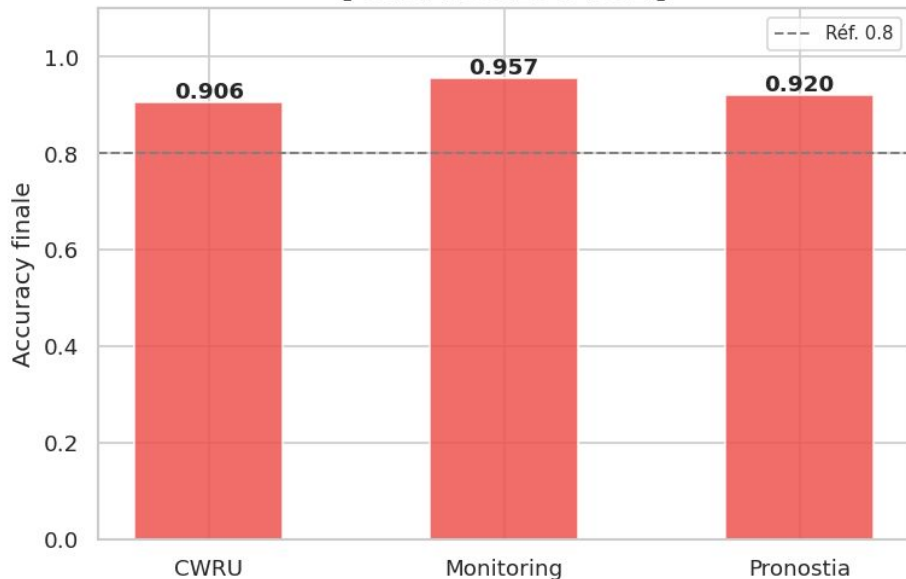
Sprint 19 — HDC : per-task accuracy par dataset [Board NUCLEO-F439ZI]



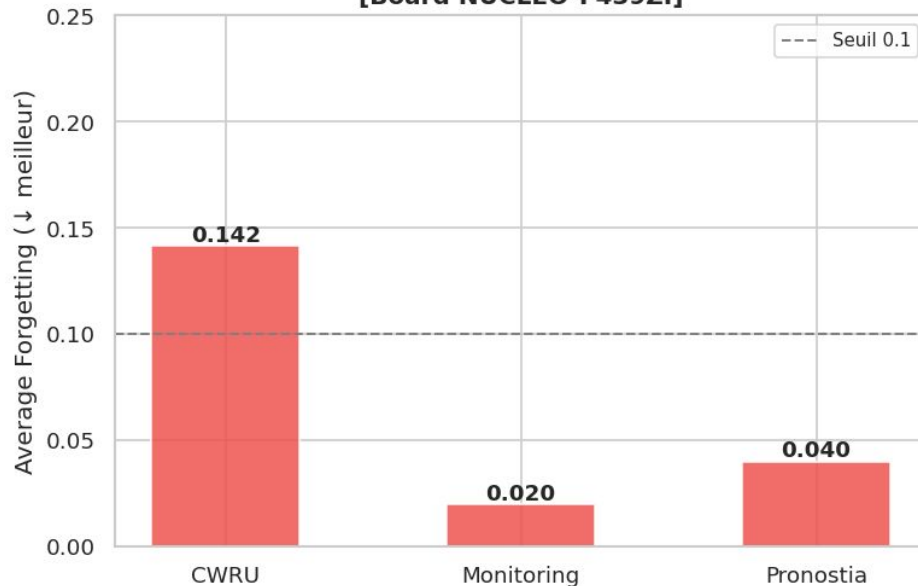
Analyse des résultats des expériences _ TinyOL

Sprint 19 — TinyOL cross-dataset (board) — exp_S19_10 / S19_11 / S19_12

TinyOL — Accuracy finale par dataset
[Board NUCLEO-F439ZI]



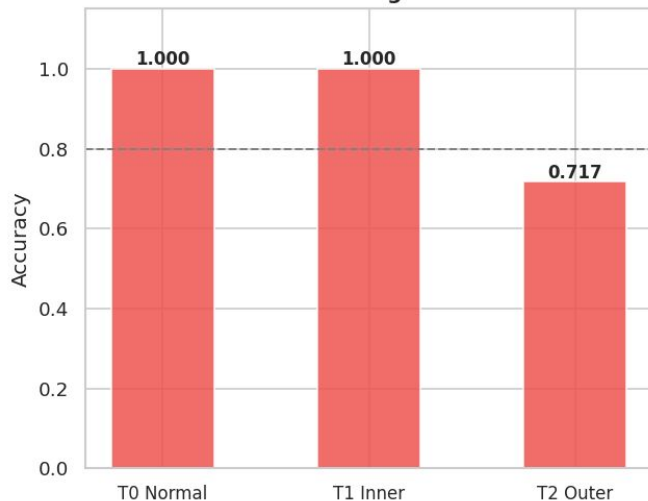
TinyOL — Average Forgetting par dataset
[Board NUCLEO-F439ZI]



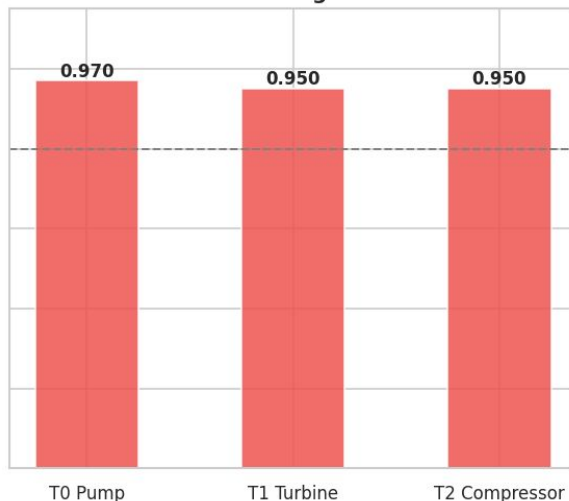
Analyse des résultats des expériences _ TinyOL

Sprint 19 — TinyOL : per-task accuracy par dataset [Board NUCLEO-F439ZI]

TinyOL — CWRU
acc=0.906 fgt=0.142



TinyOL — Monitoring
acc=0.957 fgt=0.020

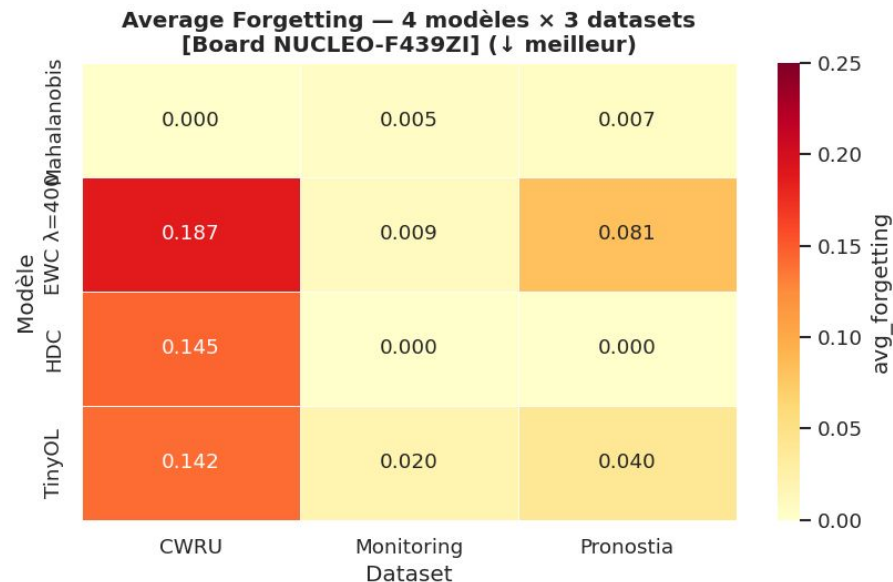
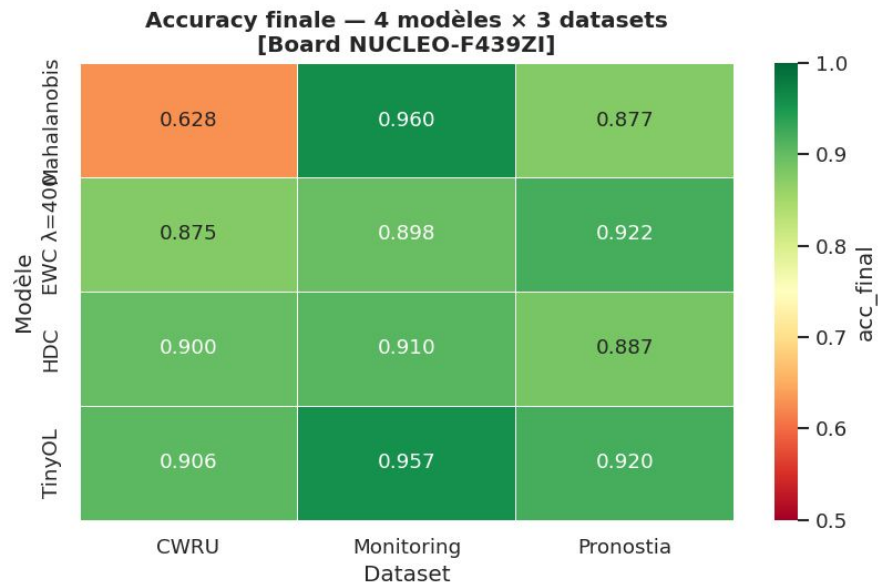


TinyOL — Pronostia
acc=0.920 fgt=0.040

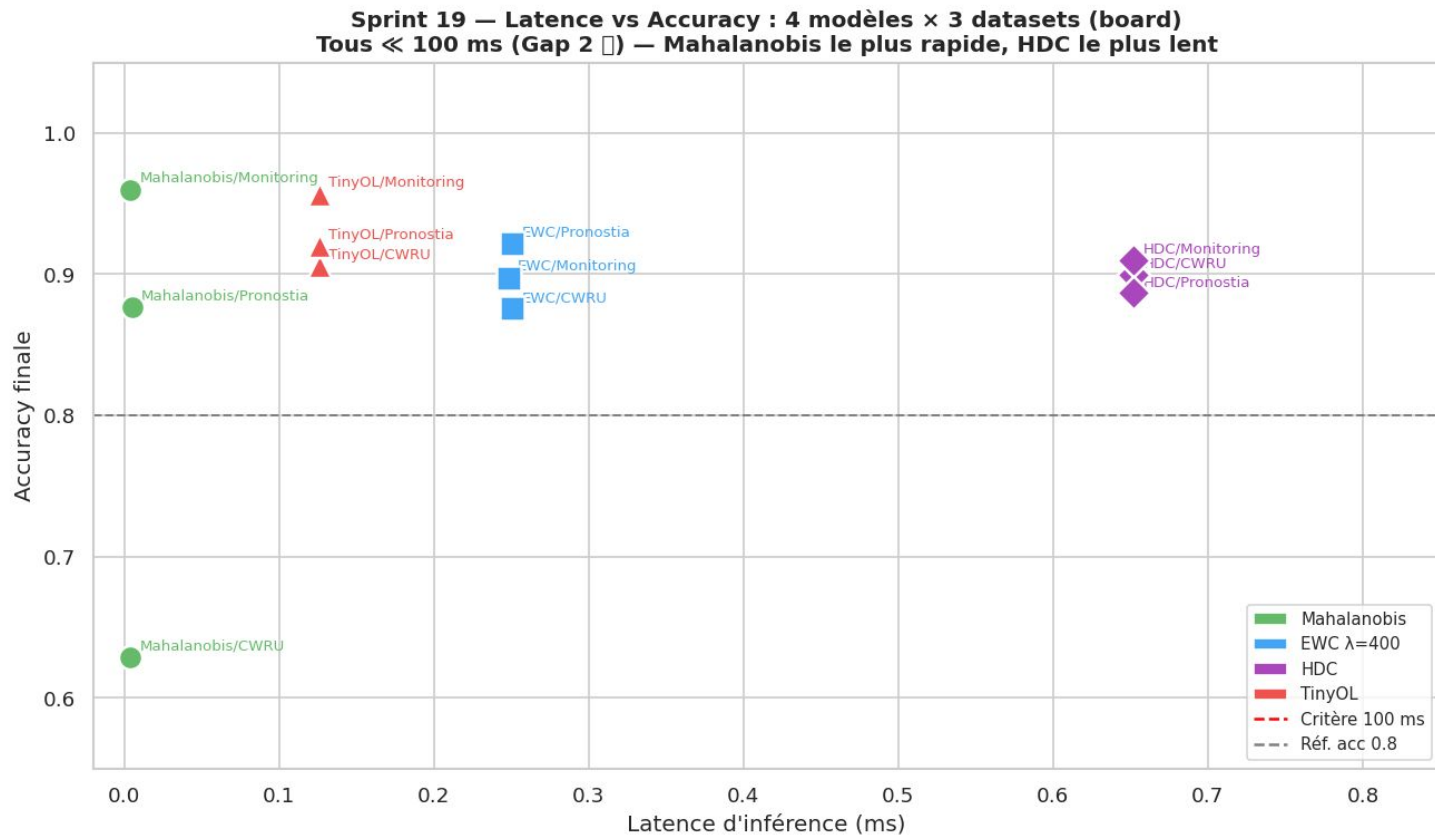


Analyse des résultats des expériences _ Recap

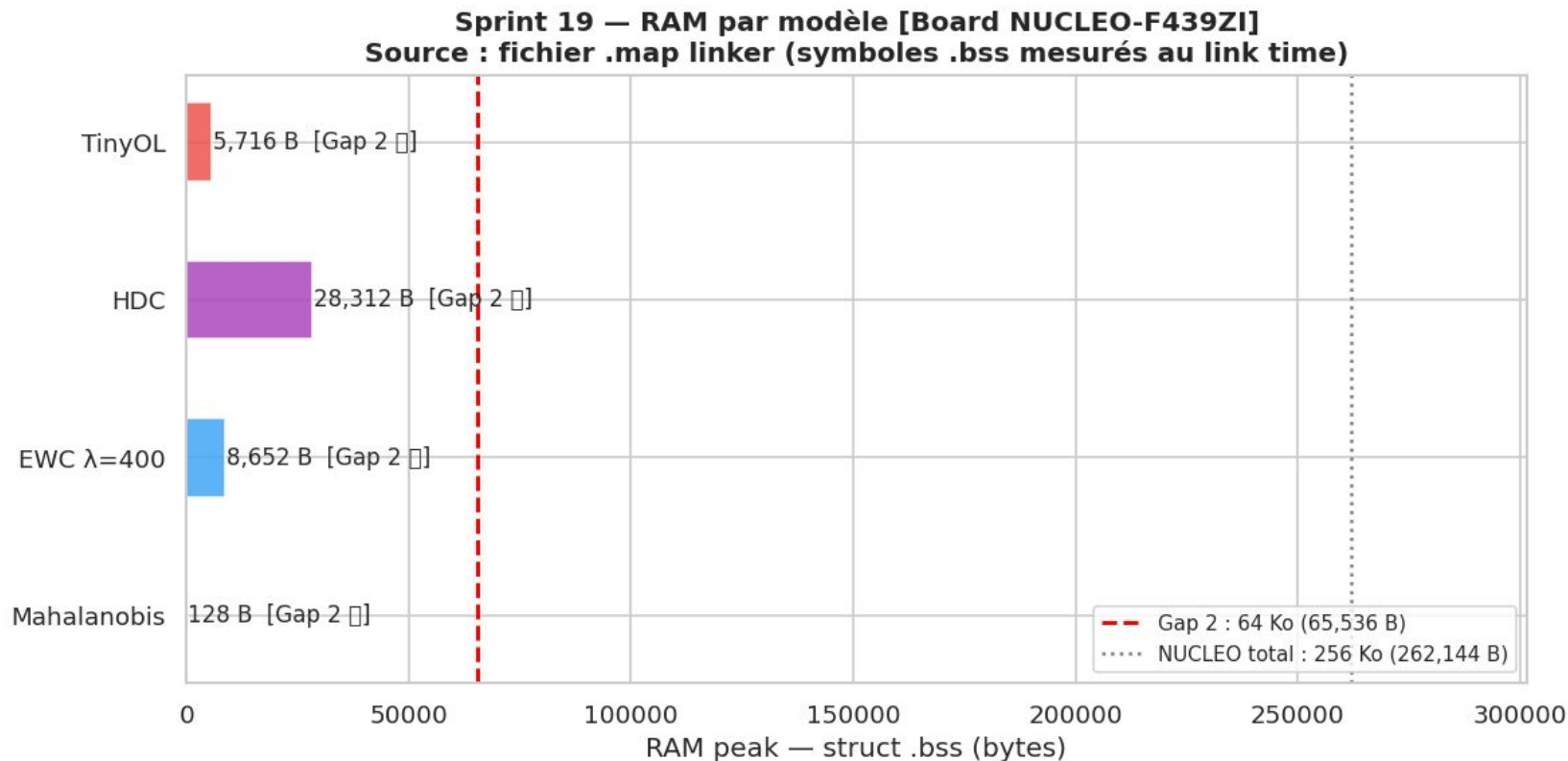
Sprint 19 — Vue heatmap complète : board NUCLEO-F439ZI



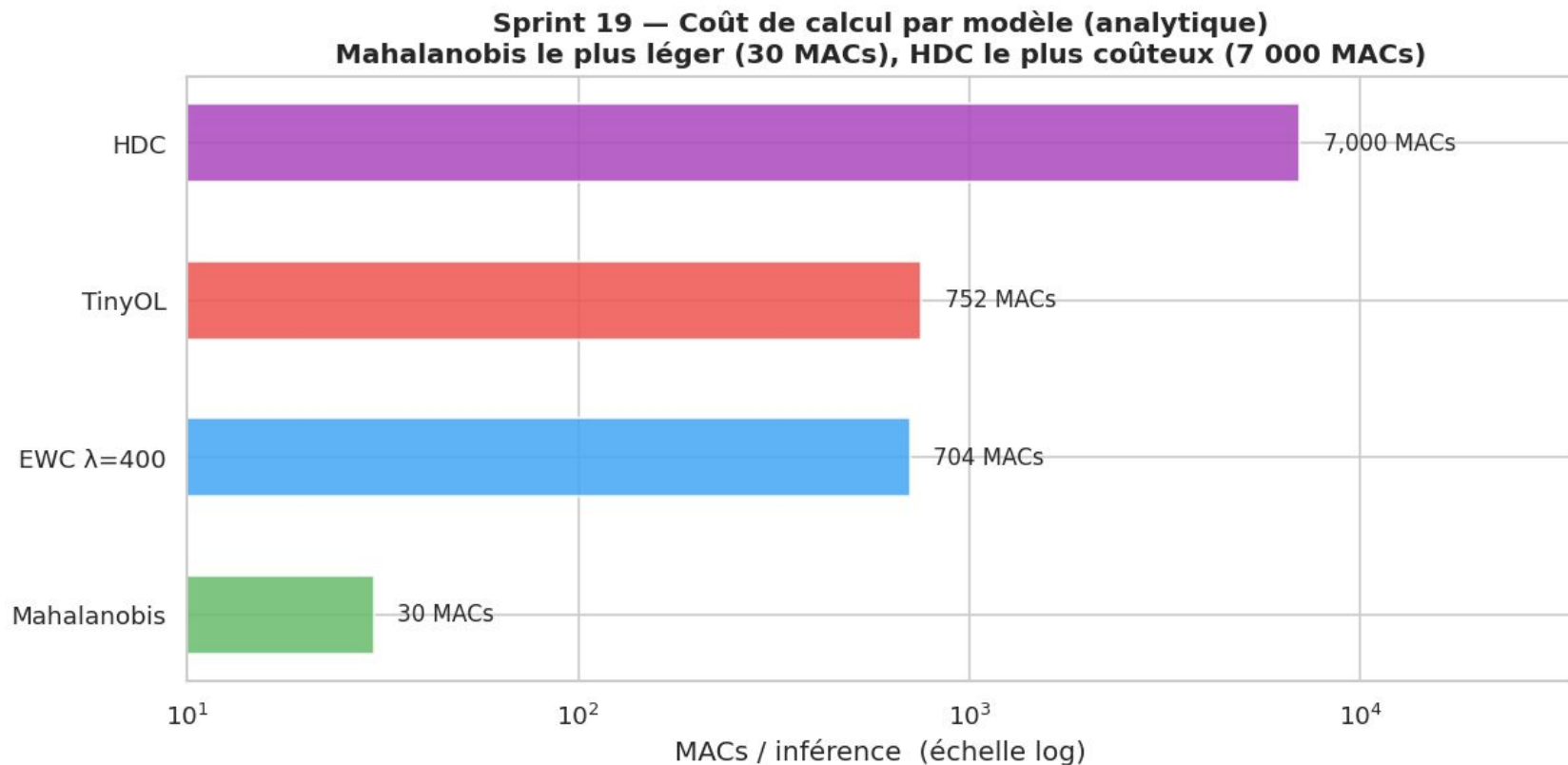
Analyse des résultats des expériences Latence vs Acc



Analyse des résultats des expériences RAM par modèle



Analyse des résultats des expériences MACs par modèle



Pistes pour la suite

Introduction de 2 Nouveau datasets (CMAPSS et Paderborn)

Évaluer la possibilité de faire marcher multiple modèles en parallèles sur la carte.

-> Combiner les capacités des modèles.

Evaluer d'autre type de tâche sur la carte

Prediction RUL.

Implémenter un protocole expérimental.